

Supplementary Material (Multi-Agent Dual-Path Verification Framework for Scalable Graph Reasoning)

A Per-Benchmark Detailed Results

The following tables disaggregate the main-paper accuracies by task category. The “Previous SOTA” column follows the strongest comparison values used in the corresponding benchmark-level analysis. Brief observations highlight the main gains and residual failure modes.

A.1 GraphWiz

Table 1: Task-level accuracy (%) on GraphWiz.

Task	GPT	Previous SOTA	DuVerG
Cycle	82.0	100.0	99.2
Connect	64.8	86.5	100.0
Bipartite	67.3	96.3	100.0
Topology	100.0	100.0	95.0
Shortest	8.5	95.0	100.0
Triangle	10.3	94.0	89.2
Flow	19.3	94.0	97.5
Hamilton	36.3	32.5	99.2
Subgraph	40.3	99.3	100.0
Avg	47.6	88.6	97.8

Observation. Gains are strongest on shortest path, flow, Hamiltonian, and subgraph queries, while cycle, topology, and triangle counting retain visible headroom.

A.2 NLGraph

Table 2: Task-level accuracy (%) on NLGraph.

Task	GPT	Previous SOTA	DuVerG
Connect	72.0	97.0	100.0
Cycle	52.9	100.0	100.0
Topology	20.0	94.8	97.8
SP	35.9	98.4	100.0
MaxFlow	98.3	100.0	98.3
Bipartite	29.8	100.0	98.8
Hamilton	22.4	100.0	100.0
GNN	0	97.4	100.0
Avg	51.4	97.8	99.5

Observation. NLGraph is close to saturation for stronger systems; the remaining margin mainly reflects robustness across task formulations.

A.3 Talk like a Graph

Table 3: Task-level accuracy (%) on Talk like a Graph benchmark.

Task	GPT	Previous SOTA	DuVerG
NCount	100.0	100.0	99.3
ECount	45.6	99.2	100.0
EExist	87.0	61.0	100.0
NDeg	54.6	98.6	99.3
CNodes	50.2	99.2	100.0
Cycle	92.6	99.4	100.0
SP	45.6	99.1	100.0
TriCount	14.4	99.2	99.3
DiscNodes	41.3	54.0	84.7
MaxFlow	36.7	77.3	98.0
NClass	51.3	74.0	100.0
Reach	97.3	98.0	100.0
Avg	59.7	88.3	98.4

Observation. DuVerG consistently improves over the previous SOTA on diverse structural tasks, with notable gains on edge existence, max-flow, node classification, and disconnected-node reasoning. Near-saturated counting and reachability tasks leave limited room for further improvement.

A.4 GraphInstruct

Table 4: Task-level accuracy (%) on GraphInstruct.

Task	GPT	Previous SOTA	DuVerG
Neighbor	98.0	100.0	99.0
Bipartite	89.7	99.0	100.0
Edge	90.7	99.3	94.7
Connectivity	92.2	100.0	99.7
Degree	98.0	100.0	99.7
DFS	89.3	95.7	97.0
Predecessor	47.4	98.0	99.7
TopoSort	59.0	98.3	99.7
ConnComp	39.3	94.7	99.7
CommonNbr	49.0	100.0	99.0
Hamilton	33.7	97.7	100.0
Jaccard	17.0	100.0	99.0
SP	57.3	100.0	100.0
Diameter	23.0	100.0	99.7
PageRank	15.3	16.3	91.7
MaxFlow	15.0	99.3	100.0
MST	1.0	0	100.0
Avg	53.7	88.1	98.7

Observation. The largest gains appear on PageRank and MST; near-ceiling structural tasks show much smaller differences.

A.5 GraphArena

Table 5: Task-level accuracy (%) on GraphArena.

Task	GPT	Previous SOTA	DuVerG
Connect	56.1	100.0	95.1
Diameter	32.3	0.0	95.0
Distance	61.8	92.4	99.3
GED	20.0	3.1	38.0
MCP	29.6	98.0	99.7
MCS	50.7	12.7	86.0
MIS	26.9	68.3	90.6
MVC	14.7	9.7	49.3
Neighbor	82.0	98.8	99.8
TSP	14.3	23.0	63.7
Avg	39.2	50.6	81.7

Observation. GraphArena shows strong gains on several combinatorial categories, while GED, MVC, and TSP remain the main residual difficulties.

A.6 OGB Benchmarks

Table 6: Task-level accuracy (%) on the OGB-derived benchmarks.

Dataset / Task	GPT	Previous SOTA	DuVerG
<i>ogbn-arxiv</i>			
TriCount	13.5	99.2	100.0
ClustCoef	14.6	83.5	100.0
3-Reach	44.2	99.3	98.1
PageRank	15.5	30.3	76.2
Avg	22.4	79.0	93.5
<i>ogbl-ppa</i>			
TriCount	3.2	100.0	100.0
ClustCoef	3.8	88.0	100.0
3-Reach	30.9	85.4	83.8
PageRank	3.2	21.1	47.8
Avg	10.5	74.6	82.6

Observation. Projection and symbolic execution are strongest on local counting and clustering; 3-hop reachability slightly favors the previous SOTA, and local PageRank remains the hardest OGB task.

B Reproducibility and Implementation Details

These control/resource constants were selected on preliminary development queries before test evaluation and then

fixed across benchmarks.

B.1 Fixed Hyperparameters

Table 7: Fixed control and resource settings used by DuVerG.

Category	Hyperparameter	Value
Routing	Large-graph threshold θ_{large}	2000
	Density threshold θ_ρ	0.05
	Projection scale trigger θ_{auto}	2000 & 100
	& θ_{seed}	
LPSP	Default hop k	2
	Min. node budget $B_v^{\min}$	2000
	Max. node budget $B_v^{\max}$	2×10^6
	Max. edge budget $B_e^{\max}$	10^8
	Base branching factor b_{base}	50
LLM	Router / Planner / Critic temperature	0.1
	Coder A / B temperature	0.2
	Reasoner T_1	0.2
	Reasoner T_2	0.6
Exec.	Sandbox timeout	300 s
	Task timeout	300 s
	Max. repair iterations M	2

Routing thresholds protect path selection, LPSP bounds the retained view, temperatures induce controlled diversity, and execution limits bound recovery cost. No row is tuned separately for ogbn-arxiv or ogbl-ppa.

B.2 Evaluation Inputs and Task Metadata

Each raw benchmark record is split into an inference object and an evaluator-only reference before the controller is invoked. The inference object contains the natural-language query, graph input, dataset identifier, and a task-family label normalized from the benchmark’s public task specification. The reference answer is excluded from this object and is supplied to the evaluator only after inference terminates. Consequently, neither routing, candidate generation, verification, nor repair can observe evaluation correctness.

The **TaskType** field shown in the prompt templates below denotes this frozen public task-family label and its output schema; it is not a symbolic/neural path label. The Router still selects the path from the query, graph statistics, lexical guardrails, and its structured model output. Task-family entries are prepared before test evaluation from public task definitions and graph-library documentation. They contain admissible algorithm families, common edge cases, and output conventions, but no test-instance identifiers, graph contents, reference answers, solution traces, or observed test outcomes. The knowledge-free ablation in the main paper removes these entries while leaving the remaining controller and evaluation protocol unchanged.

B.3 On-Disk LPSP Semantics

DuVerG separates the graph-storage backend from the bounded graph view supplied to the reasoning path. In the reported OGB experiments, a plain or compressed edge list is parsed once into a cached adjacency store. LPSP subsequently requests adjacency only for the current multi-source BFS frontier, validates query-mentioned seeds, and enforces the configured hop, node, and edge budgets before constructing the retained view. Only this retained view is materialized as a NetworkX object or exposed to downstream agents; the full graph is neither converted into a NetworkX object nor serialized into an LLM context.

The streaming description in the main paper refers to these frontier-restricted access semantics rather than requiring a particular storage index. The reported backend uses an adjacency cache to amortize repeated queries,

while the same LPSP procedure can be paired with a scan-backed or external-memory adjacency index without changing seed validation, expansion order, or budget enforcement. The cache is therefore part of graph access, not part of the projected reasoning context measured in the node, edge, token, and runtime analyses. If no valid anchor remains, projection is bypassed rather than replaced by a hub-centered surrogate; global queries likewise retain the full-graph execution path. The in-memory variant applies the implementation edge score when its induced edge set exceeds the configured cap.

B.4 Verification and Repair Semantics

Each evaluation item invokes the controller once. As specified above, its reference answer remains outside the inference task object, so evaluation correctness never triggers another attempt or supplies repair feedback. Verification is status-aware: normalized agreement permits early exit without certification, and one successful execution is retained when the other fails. When two successful executions disagree, the Critic performs a focused check over the two existing outputs and program traces. If that check remains unresolved, the controller switches to the neural path once; a neural conflict that remains unresolved returns a best-effort output with an unsuccessful status. Only execution exceptions and timeouts can enter the bounded repair loop of at most M iterations. No third solver or third program is introduced.

C Prompt Templates

The base templates below are populated from shared state and contain no benchmark answers or solution traces. The **TaskType** placeholder and any retrieved task-family entry follow the provenance and information boundaries defined in Appendix B: both are frozen before test evaluation, and neither contains instance-level answers or test-derived feedback. Paired-agent diversity hints are fixed across datasets, while execution-error feedback is added only within the bounded repair loop described above.

C.1 Router Agent

The Router LLM receives the query, the public task-family label, and lightweight graph metadata. The task-family label provides an answer schema but does not specify a reasoning path. The Router’s preference and local-decomposition flag are combined with the fixed lexical and graph-scale guardrails, deterministic query-only seed extraction, and locally computed adaptive budgets before projection or path execution.

Prompt Template

```
system: "You are a Complexity-Aware Router for hybrid
        neural-symbolic graph reasoning."
user_template: |
    Query: {query}
    TaskType: {task_type}
    GraphStats: {graph_stats}

Decide route:
- symbolic: computation/algorithmic (shortest path/flow
    /matching/substructure/hamilton) -> prefer NetworkX
    code.
- neural: semantic/ambiguous/preference (who likes
    golf?) -> answer directly.
```

Output JSON:

```
{
  "route": "symbolic" or "neural",
  "why": "Brief reasoning",
  "need_decomposition": true/false,
  "decomposition": {
    "seed_nodes": [],
    "hop": {default_hop},
    "node_budget": {node_budget},
    "edge_budget": {edge_budget}
  }
}
```

C.2 Planner Agent

The Planner is used only on the symbolic path. It specifies an algorithm family, key steps, and defensive checks without itself producing executable code.

Prompt Template

```
system: "You are a Graph Task Planner. Analyze the
        problem and outline logical steps."
user_template: |
  Task: {query}
  TaskType: {task_type}
  GraphStats: {graph_stats}
  GraphDataHint: {graph_hint}

  Provide a high-level execution plan:
  - algorithm_family: The category of algorithm (e.g.,
    bfs, flow, backtracking, isomorphism).
  - key_steps: Logical steps to solve the problem
    programmatically.
  - defensive_checks: Edge cases to handle (e.g.,
    disconnected graph, type mismatch).

  Output JSON:
  {{
    "algorithm_family": "...",
    "key_steps": [...],
    "defensive_checks": [...]"
  }}
```

C.3 Coder Agents

The two Coders share the base template below. The controller prepends the same task entry and supplies different diversity hints: Coder A favors canonical NetworkX primitives, whereas Coder B emphasizes explicit corner-case handling. Execution-error feedback is populated only during the bounded internal repair loop.

Prompt Template

```
system: "You are an expert Python programmer
        specializing in NetworkX for Graph Theory. You write
        robust, fault-tolerant code."
user_template: |
  Task: {query}
  TaskType: {task_type}
  Plan: {plan_json}

  Graph Input Data:
  {graph_payload_hint}

  Constraints:
  1. Use ONLY standard python libraries and 'networkx'.
  2. Do NOT install new packages.
  3. PRINT only the final answer. Do not print debug
    info.
  4. WRAP dangerous code in try/except blocks and print
    a safe fallback (e.g., "No") if it fails.

  If this is a RETRY, fix the error:
```

```
{error_feedback}
```

Write the complete Python code in a single `'''python'''` block.

C.4 Reasoner Agents

The neural pair shares a prompt and graph view. Fixed temperatures reported in Appendix B pair a conservative completion with a more exploratory one.

Prompt Template

```
system: "You are a Logical Graph Reasoner. Solve the
        problem step-by-step using semantic reasoning."
user_template: |
  Task: {query}
  TaskType: {task_type}
  GraphSummary: {graph_hint}

  Perform chain-of-thought reasoning and provide the
  final answer.

  Answer:
```

C.5 Critic Agent

The Critic receives candidate outputs together with execution status, performs answer-type-aware normalization, and separates valid negative/empty answers from execution failures. Agreement allows early exit without certification; successful disagreement requests a concrete check.

Prompt Template

```
system: "You are a strict execution-grounded verifier.
        Compare two candidate outputs under the query's
        answer type and execution status."
consistency_template: |
  Query: {query}
  AnswerType: {answer_type}
  StatusA: {status_a}
  OutputA: {out_a}
  StatusB: {status_b}
  OutputB: {out_b}

  1. Normalize only format differences that preserve the
    task answer.
  2. If both successful outputs are equivalent after
    normalization, output consistent=true.
  3. If exactly one execution failed or timed out, prefer
    the successful candidate.
  4. Never interpret a legitimate "No", "False", or empty
    answer as failure solely from its text.
  5. If both executions succeed but disagree, identify
    one concrete condition for a focused check.
```

Output JSON:

```
{
  "consistent": true/false,
  "why": "Brief explanation",
  "resolution": "pick_a" or "pick_b" or "
    need_tiebreaker",
  "tiebreaker_hint": "Explain what specific check is
    needed to verify the truth."
}
```

Focused Tiebreaker Check

The tiebreaker verifies the two existing traces rather than generating a third solution. It checks only the concrete condition identified from the disagreement.

Prompt Template

```
focused_check_template: |
  Query: {query}
  AnswerType: {answer_type}
```

```
CandidateA: {out_a}  
CandidateB: {out_b}  
Check: {tiebreaker_hint}  
RelevantTraceA: {trace_a}  
RelevantTraceB: {trace_b}
```

```
Evaluate only the requested condition against the two  
existing candidates.  
Do not generate a third solution.  
Return: pick_a, pick_b, or unresolved.
```

Status and Fallback Handling

Execution status is separate from answer content, so “No,” “False,” or an empty set remain valid outputs. Exceptions and timeouts use the bounded repair loop, whereas unresolved successful disagreements switch paths without generating another program. A conflict that remains unresolved after neural reconciliation is returned only as an explicitly unsuccessful best-effort output.